

GradeAI

AI-Powered Assignment Grading System

Automated Grading via Google Classroom & Gemini AI

Field	Details
Authors	ShahZaib (UL-BSCSE-23-40) Sajid Hussain (UL-BSCSE-23-14)
University	University of Layyah
Department	Computer Science
Course	Artificial Intelligence
Semester	6th Semester
Submitted To	Sir Faisal Hafeez
Date	June 2026

Table of Contents

#	Section	Page
1	Introduction & Objectives	3
2	System Architecture	3
3	Technologies Used	4
4	Implementation Details	4
4.1	Google OAuth 2.0 Authentication	4
4.2	Classroom & Drive API Integration	5
4.3	File Content Extraction	5
4.4	AI Grading with Gemini	5
4.5	Grade Push-Back to Classroom	6
4.6	Frontend Dashboard	6
5	OAuth Scopes & Permissions	7
6	Challenges & Solutions	7
7	Results & Outcomes	8
8	Future Work	8
9	Conclusion	8
10	References	9

1. Introduction & Objectives

The manual grading of student assignments is one of the most time-consuming responsibilities for educators. As class sizes grow and submission volumes increase, teachers spend significant hours reviewing assignments, providing feedback, and recording grades — time that could be better invested in direct instruction and student support.

GradeAI is an AI-powered assignment grading system developed as part of the Artificial Intelligence course at the University of Layyah. It integrates Google Classroom as the submission source, uses the Google Gemini 2.5 Flash language model to evaluate student work, and automatically pushes grades back to Google Classroom — reducing the manual grading workload to near zero.

Objectives

- Automate the fetching of student submissions from Google Classroom via the Classroom API.
- Extract readable content from student-submitted PDFs and Google Docs using the Drive API.
- Apply Gemini AI to evaluate submissions against the assignment description and produce structured grades.
- Automatically push computed grades back to Google Classroom with proper teacher workflow (draft → return).
- Present results in a clean, informative web dashboard for teacher review.
- Detect potential plagiarism and flag suspicious submissions.

2. System Architecture

GradeAI follows a layered Flask web application architecture with clear separation of concerns across authentication, API services, AI processing, and presentation layers.

Layer	Component	Responsibility
Presentation	Jinja2 Templates + CSS/JS	Teacher-facing dashboard & grading results UI
Routing	Flask Blueprints (auth, dashboard, grading)	HTTP request handling & session management
Service	google_auth.py	OAuth 2.0 flow, token storage & refresh
Service	classroom.py	Classroom API: fetch courses, submissions, push grades
Service	file_extractor.py	PDF & Google Doc text extraction via Drive API
Service	gemini_grader.py	Gemini AI prompt construction & grade parsing
External APIs	Google Classroom API v1	Source of truth for courses, assignments, submissions
External APIs	Google Drive API v3	File download for submitted documents

External APIs	Google Gemini 2.5 Flash	Large language model for AI-based grading
---------------	-------------------------	---

Table 1: System Architecture Layers

Project File Structure

```
ai_grading_system/  
  app.py           # Flask app factory & entry point  
  config.py        # OAuth scopes & environment config  
  requirements.txt  
  client_secret.json  # Google OAuth credentials  
  services/  
    google_auth.py  # OAuth helpers  
    classroom.py    # Classroom & Drive API  
    file_extractor.py  
    gemini_grader.py  
  routes/  
    auth.py  dashboard.py  grading.py  
  templates/  
    base.py
```

3. Technologies Used

Technology	Version	Purpose
Python	3.11+	Core programming language
Flask	3.x	Web framework & routing
Google Gemini API	2.5 Flash	AI language model for grading
Google Classroom API	v1	Fetch submissions, push grades
Google Drive API	v3	Download submitted files
google-auth-oauthlib	Latest	OAuth 2.0 flow management
google-api-python-client	Latest	Google API client library
PyMuPDF / pdfplumber	Latest	PDF text extraction
Jinja2	3.x	HTML templating
python-dotenv	Latest	Environment variable management

Table 2: Technology Stack

4. Implementation Details

4.1 Google OAuth 2.0 Authentication

Authentication is handled via Google's OAuth 2.0 protocol using the **google-auth-oauthlib** library. When a teacher visits the application, they are redirected to Google's consent screen where they grant the necessary permissions. After successful consent, Google returns an authorization code which the application exchanges for an access token and refresh token stored in the Flask session.

The OAuth flow is configured with **access_type='offline'** and **prompt='consent'** to ensure a refresh token is always issued, enabling the application to make API calls without requiring the teacher to re-authenticate on every request. Credentials are serialized to a dictionary and stored in the encrypted Flask session between requests.

```
# routes/auth.py – OAuth initiation
authorization_url, state = flow.authorization_url(
    access_type='offline',
    include_granted_scopes='true',
    prompt='consent'
)
```

4.2 Google Classroom & Drive API Integration

The **classroom.py** service module wraps all Google API interactions. It uses the **googleapiclient.discovery** library to build typed service clients for both the Classroom API (v1) and Drive API (v3). Key operations include:

- **get_courses()** — Lists all active courses where the authenticated user is a teacher.
- **get_coursework()** — Retrieves all assignments for a given course.
- **get_submissions()** — Fetches all student submissions for an assignment, filtering to TURNED_IN and RETURNED states.
- **get_student_profile()** — Resolves a student user ID to a full display name.
- **download_drive_file()** — Streams a Drive file into memory using MediaIoBaseDownload.
- **post_grade_to_classroom()** — Patches draftGrade, then calls return_() to publish.

4.3 File Content Extraction

Student submissions arrive in different formats depending on how they were submitted. The **file_extractor.py** module handles format detection and text extraction:

- **Google Docs** — Exported as plain text via the Drive API export endpoint.
- **PDF files** — Downloaded from Drive and parsed using PyMuPDF or pdfplumber to extract plain text.
- **Plain text files** — Read directly from the Drive file download stream.

The extracted text is passed to the Gemini grader as the student's answer. If no readable content can be extracted from a submission, the system records an error for that student and continues grading other submissions.

4.4 AI Grading with Gemini 2.5 Flash

The core intelligence of GradeAI lives in **gemini_grader.py**. For each submission, a structured prompt is constructed containing the assignment title, assignment description, and the student's extracted text.

Gemini is instructed to respond exclusively in JSON format with the following schema:

```
{
  "marks": 91,
  "grade_letter": "A+",
  "overall_feedback": "...",
  "rubric": {
    "content_accuracy": { "score": 30, "max": 30, "comment": "..." },
    "clarity": { "score": 20, "max": 25, "comment": "..." },
    "...",
  }
  "strengths": ["...", "..."],
  "improvements": ["...", "..."],
  "plagiarism_flag": false
}
```

The response is parsed with `json.loads()` after stripping any markdown fences. The grader implements a retry mechanism — if the model returns a non-200 response or malformed JSON, it retries up to 3 times before recording a grading failure for that submission. The `marks_to_grade()` helper converts a numeric score to a letter grade (A+, A, B, C, D, F) for display.

4.5 Grade Push-Back to Google Classroom

Pushing grades back to Classroom requires a precise two-step API flow enforced by Google. Simply setting `assignedGrade` directly causes a **403 ProjectPermissionDenied** error. The correct teacher workflow is:

- **Step 1 — PATCH draftGrade:** The submission is patched with `updateMask='draftGrade'`, setting an internal draft that is not yet visible to the student.
- **Step 2 — return_():** The return endpoint is called, which publishes the draft grade as the `assignedGrade` and sends a notification to the student.

```
# services/classroom.py – post_grade_to_classroom()
# Step 1: set draft grade
service.courses().courseWork().studentSubmissions().patch(
    courseId=course_id, courseWorkId=work_id,
    id=submission_id, updateMask='draftGrade',
    body={'draftGrade': float(marks)}
).execute()
# Step 2: publish to student
service.courses().courseWork().studentSubmissions().return_(
    courseId=course_id, courseWorkId=work_id,
    id=submission_id, body={}
).execute()
```

4.6 Frontend Dashboard

The grading results page is rendered as a single-page Jinja2 template with inline CSS and JavaScript. It presents a dark-themed, responsive dashboard with:

- A **statistics bar** showing total submissions, graded count, posted count, average, highest, and lowest scores.
- **Per-student cards** with an SVG score ring that visually represents the grade percentage with colour coding (green ≥ 80 , amber ≥ 60 , orange ≥ 40 , red below 40).
- A **rubric breakdown table** with per-criterion scores, mini progress bars, and pass/fail status indicators.
- **Strengths and Improvements** columns rendered side by side for each student.
- A **plagiarism flag badge** displayed prominently on suspicious submissions.
- A **'Posted to Classroom'** badge confirming successful grade push, or a warning badge if posting failed.
- A **copy-to-clipboard** button that formats the student's full rubric as plain text.

5. OAuth Scopes & Permissions

The application requests the following OAuth 2.0 scopes from the authenticating teacher. All scopes must be registered on the Google Cloud OAuth consent screen for the grade write operations to succeed.

Scope	Access Type	Purpose
openid	Auth	Identity verification
userinfo.email	Read	Teacher email address
userinfo.profile	Read	Teacher name & profile picture
classroom.courses.readonly	Read	List active courses
classroom.coursework.students	Read/Write	Fetch & grade student submissions
classroom.coursework.me	Read	Access teacher's own coursework
classroom.student-submissions.students.readonly	Read	Read student submission data
classroom.rosters	Read	Resolve student names from user IDs
drive.readonly	Read	Download submitted PDF & Doc files

Table 3: Required OAuth Scopes — Row highlighted in amber is critical for grade write-back.

6. Challenges & Solutions

Challenge 1: 403 ProjectPermissionDenied on Grade Push

Problem: The Classroom API returned a 403 error when attempting to write grades, even with the correct scope declared in code. This was caused by the OAuth consent screen being in 'Testing' mode and the write scopes not being formally added to the consent screen configuration in Google Cloud Console.

Solution: Published the OAuth app to 'In Production' status, formally registered all scopes on the consent screen, and cleared cached tokens to force re-authentication with the updated permissions.

Challenge 2: Correct Grade Posting API Flow

Problem: Initially, setting assignedGrade directly via PATCH caused permission errors on unverified projects. The Google Classroom API enforces a teacher workflow that does not allow bypassing the draft state.

Solution: Implemented the correct two-step flow: PATCH with updateMask='draftGrade' first, followed by a separate return_() call to publish the grade to the student.

Challenge 3: Gemini Response Parsing

Problem: Gemini occasionally wraps its JSON response in markdown code fences (``json ... ``) rather than returning raw JSON, causing json.loads() to fail.

Solution: Added a pre-processing step that strips markdown fences before parsing, with a retry mechanism that attempts up to 3 calls before recording a grading failure.

Challenge 4: Varied Submission File Formats

Problem: Students submit assignments in different formats — Google Docs, PDFs, and plain text — each requiring a different extraction strategy.

Solution: Built a unified file_extractor.py module that detects the MIME type of each attachment and routes to the appropriate extraction method, returning normalized plain text in all cases.

7. Results & Outcomes

GradeAI was tested on a live Google Classroom course with real student submissions. The system successfully completed the full pipeline from login to grade publication for all turned-in submissions.

Metric	Result
Submissions processed	All TURNED_IN and RETURNED submissions fetched automatically
AI grading accuracy	Rubric scores consistent with manual teacher evaluation
Grade push success rate	100% after OAuth consent screen was correctly configured
Average grading time per student	Under 5 seconds per submission (Gemini 2.5 Flash)

Plagiarism detection	Flagged submissions with identical or near-identical content
Dashboard load time	Under 2 seconds for courses with up to 30 submissions

Table 4: Testing Results Summary

8. Future Work

- **Batch grading queue** — Process large classes asynchronously with a Celery/Redis task queue to avoid request timeouts.
- **Custom rubric builder** — Allow teachers to define their own grading criteria and weightings via the dashboard UI.
- **Multi-assignment grading** — Grade all assignments across a course in a single operation.
- **Grade analytics** — Add course-level and student-level trend charts showing performance over time.
- **Feedback delivery** — Integrate with Gmail API to email detailed Gemini feedback reports directly to students.
- **Support for more file types** — Extend the extractor to handle DOCX, PPTX, and image-based submissions via OCR.
- **Deployment** — Package the application for production deployment on Google Cloud Run with proper HTTPS and secret management.
- **Model comparison** — Benchmark grading consistency across different Gemini model versions and alternative LLMs.

9. Conclusion

GradeAI demonstrates a practical and effective application of large language models to an important real-world educational problem. By combining the Google Classroom API for submission retrieval and grade publishing with the Gemini 2.5 Flash model for intelligent evaluation, the system creates a complete automated grading pipeline that requires minimal teacher intervention.

The project highlights several important lessons in AI system design: the importance of structured prompt engineering for reliable JSON output, the complexity of OAuth 2.0 scope management for write operations, and the value of robust error handling when integrating multiple third-party APIs in a single pipeline.

Beyond reducing grading time, GradeAI produces richer, more consistent feedback than time-pressed manual grading allows — providing each student with a detailed rubric breakdown, specific strengths, and targeted areas for improvement. This project establishes a strong foundation for future work in AI-assisted education technology.

10. References

[1] Google LLC. (2024). *Google Classroom API Reference*. <https://developers.google.com/classroom>

- [2] Google LLC. (2024). *Google Drive API Reference*. <https://developers.google.com/drive>
- [3] Google LLC. (2024). *Gemini API Documentation*. <https://ai.google.dev/docs>
- [4] Google LLC. (2024). *OAuth 2.0 for Web Server Applications*. <https://developers.google.com/identity/protocols/oauth2/web-server>
- [5] Pallets Projects. (2024). *Flask Documentation*. <https://flask.palletsprojects.com>
- [6] Python Software Foundation. (2024). *Python 3.11 Documentation*. <https://docs.python.org/3.11>
- [7] Google LLC. (2024). *google-auth-oauthlib Library*. <https://google-auth-oauthlib.readthedocs.io>
- [8] PyMuPDF Contributors. (2024). *PyMuPDF Documentation*. <https://pymupdf.readthedocs.io>